
Lab Session gpg

Ce TP est basé sur la documentation réalisée par Mike Ashley (<https://www.gnupg.org/gph/fr/manual.html>), et a deux objectifs :

- Comprendre les principes de base du chiffrement asymétrique ;
- Analyser le fonctionnement du protocole de sécurité PGP

Présentation. Au contraire d'openSSL, dont le but est la transmission de données entre 2 personnes, le but de GPG est d'établir la confiance en l'identité réelle des personnes avec lesquelles on communique.

Chaque identité (une adresse email, par exemple) a un état et est représentée par sa *clef primaire*. On peut ajouter des sous-clefs pour gérer différents usages. Il est conseillé que chaque adresse email que vous utilisez ait une clef primaire. Toutes les opérations que l'on fait sur son état sont signées par la clef primaire. Les opérations possibles sont classées en 3 grandes catégories :

1. la création de clefs et de certificats pour ses clefs ;
2. la communication de *certificats* qui associent une clef à une identité réelle ;
3. l'utilisation de clefs pour des communications sécurisées.

Exercice 1 : Création de clefs

La commande suivante permet de créer une clef primaire pour une nouvelle identité (la réponse de gpg est dans le Fig. ??) :

```
gpg --gen-key
```

Il faut voir cette commande comme la création d'un nouveau compte dans la communauté GPG qui permettra de gérer ses accès. Le mot de passe demandé sert à chiffrer le fichier sur le disque dur. GnuPG crée différents types de paires de clés. La clef primaire doit seulement être capable de signer.

Structure des clefs : La structure d'une clef est définie récursivement :

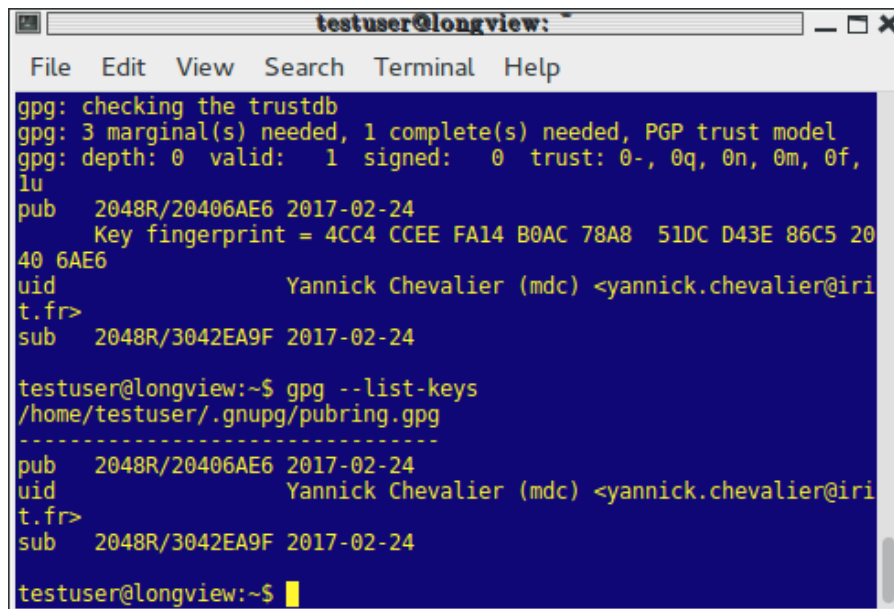
- chaque clef a un ensemble d'*identifiants uid*, qui sont les identités du propriétaire de la clef
- chaque clef a un ensemble de sous-clefs.
- uid identifiant

Taille des clefs. Ensuite, vous devez déterminer la taille de vos clés, ou plus précisément du N modulo lequel tous les calculs seront faits. Plus la clé est longue, plus elle sera résistante face à une attaque mais elle nécessite un temps plus long de traitement. Le temps d'attaque pour des clefs de n bits est à peu près $2^{7*\sqrt[3]{n}}$, soit par exemple à peu près 2^{70} pour $n = 1000$. On considère actuellement que 2048 bits est un minimum pour avoir de la sécurité pour quelques années.

Date d'expiration. Vous devez après fournir une date d'expiration. La date d'expiration doit être choisie avec soin, car il est difficile de faire parvenir la clé avec la date mise à jour aux utilisateurs qui possèdent déjà votre clé publique, mais en cas de perte de clef, il est aussi difficile de faire parvenir le certificat de révocation (voir plus bas) aux autres utilisateurs. Pour ce TP, choisissez l'option 0.

GnuPG répond en donnant des informations sur la clé générée, en particulier son identité (20406AE6 dans la Fig. 1) et son type (2048 bits, RSA). La clé est stockée dans une base de donnée (trustdb), qui stocke pour chaque des statistiques telles que :

- On verra plus tard en détail la signification de **PGP Trust model**, mais ce modèle de confiance repose sur le fait que d'autres personnes certifient (en signant avec leur clef un certificat) que la clef que vous venez de créer est bien reliée à l'identité que vous lui avez donnée ;
- La profondeur (*depth*) mesure l'éloignement d'une clef stockée à celles que vous avez créées ;
- il y a ensuite le nombre de clefs que vous avez certifié directement (profondeur 1), puis le nombre de clefs qui ont été certifiées par ces clefs (profondeur 2), etc. Une clef est valide quand elle a été suffisamment signée. Le reste de la ligne représente la qualité (d'après vous) des personnes ayant certifié les clefs (voir l'exercice *Édition des clefs*).



```

testuser@longview: ~
File Edit View Search Terminal Help
gpg: checking the trustdb
gpg: 3 marginal(s) needed, 1 complete(s) needed, PGP trust model
gpg: depth: 0 valid: 1 signed: 0 trust: 0-, 0q, 0n, 0m, 0f,
lu
pub 2048R/20406AE6 2017-02-24
Key fingerprint = 4CC4 CCEE FA14 B0AC 78A8 51DC D43E 86C5 20
40 6AE6
uid Yannick Chevalier (mdc) <yannick.chevalier@iri
t.fr>
sub 2048R/3042EA9F 2017-02-24

testuser@longview:~$ gpg --list-keys
/home/testuser/.gnupg/pubring.gpg
-----
pub 2048R/20406AE6 2017-02-24
uid Yannick Chevalier (mdc) <yannick.chevalier@iri
t.fr>
sub 2048R/3042EA9F 2017-02-24
testuser@longview:~$

```

FIG. 1 : Fin de la génération de clefs avec liste des clefs

(a) (Gestion des clefs) La commande `gpg --list-keys` vous permet de lister vos clés. Toujours dans la Fig. 1, on voit qu'il y a deux clefs :

- la clef primaire, **pub**, qui ne devrait servir que pour signer des messages ;
- une clef secondaire, **sub**, qui devrait être utilisée pour le chiffrement. Il est déconseillé d'utiliser une même clef pour chiffrer et signer.

La commande `gpg --list-secret-keys` vous permet de lister vos clés secrètes.

(b) (Générer un certificat de révocation)

Si une clef est perdue (par exemple parce que quelqu'un d'autre a réussi à apprendre votre clef secrète) ou est devenue inutilisable (parce que vous avez oublié votre mot de passe), il faut indiquer à vos connaissances qu'elle ne doit plus être utilisée. On appelle cela la *révocation*. Il est conseillé de générer tout de suite le certificat de révocation, car ce ne sera plus possible une fois le mot de passe oublié. La commande est :

```
gpg --output revoke.asc --gen-revoke identificateur
```

où *identificateur* est l'identificateur de la clef. Choisissez 0, et mettez comme commentaire qu'il s'agit d'une mesure de précaution. GPG vous informe ensuite des dangers de ce certificat.

(c) (Exportation) On partage les clefs qu'on connaît en les *exportant*. L'exportation crée un fichier avec toutes les clefs, leurs certificats, et la confiance qu'on a en ces clefs. Cette opération permet de partager son état avec d'autres personnes.

```
gpg --export --output clefs_connues.gpg -a --armor
```

Rapport. Mettez dans le rapport la liste de vos clefs après avoir créé une clef maître uniquement capable de signer, et deux sous-clefs, l'une pour signer (S) seulement (avec DSA), et l'autre pour chiffrer (E) seulement (avec ElGamal). La clef maître doit être capable de signer et de créer des certificats (SC). Pour ajouter une sous-clef à une clef maître, il faut *éditer la clef* (avec `-edit-key` et ajouter deux sous-clefs (voir les commandes d'édition possibles avec `help`)).

Exercice 2 : Gestion des clefs

Pour envoyer un message chiffré à une personne ou vérifier un message signé par cette personne (Alice), il faut d'abord connaître la clef publique utilisée. Pour cela, Alice doit exporter cette clef publique, et vous l'envoyer. Vous devez vérifier que la clef est bien celle d'Alice (normalement, sans passer par Internet). Une fois que c'est fait, vous pouvez *importer* la clef publique d'Alice. Une fois qu'elle est importée, vous pouvez l'utiliser pour envoyer des messages à Alice (si vous avez une clef qui peut chiffrer pour Alice) ou vérifier les messages qui viennent d'Alice (si vous avez la clef publique de vérification de signature d'Alice). La procédure recommandée est de faire physiquement cette procédure pour être sûr que personne n'a modifié les messages sur Internet.

(a) On importe les clefs connues de Bob avec

```
gpg --import clefs_connues_bob.gpg
```

Procédure d'importation de clef :

1. on commence par ajouter une clef publique à notre base de données ;
2. on vérifie son empreinte ;
3. si l'empreinte est correcte, on signe la clef ;

(b) Ajoutez des clefs publiques d'autres étudiants en utilisant la commande :

```
gpg --import identifiant.gpg
```

(c) (Signature d'une clef importée) La commande :

```
gpg --edit-key identificateur
```

permet de travailler sur une clef qui a été importée. On interagit avec gpg en lui envoyant des commandes à exécuter sur la clef :

fpr : permet d'obtenir l'empreinte (*fingerprint*) de la clef. Cette empreinte doit correspondre exactement à une valeur publiée indépendamment par le propriétaire de la clef. On verra plus précisément la procédure à suivre dans l'exercice *Web of Trust* ;

(t,l,nr)sign : permet de signer les identifiants d'une clef ;

check : permet de voir les clefs enregistrées et qui les a signées ;

trust : permet de spécifier le niveau de confiance qu'on a en le propriétaire de la clef.

Rapport. Exportez et échangez vos clefs au sein du groupe (ou entre plusieurs groupes). Pour la suite, évitez d'envoyer toutes les clefs à tout le monde.

Exercice 3 : Chiffrer et déchiffrer des documents

Chiffrer un document nécessite d'avoir au préalable la clé publique du destinataire. La commande suivante permet de réaliser cette opération :

```
gpg --output nom_de_fichier_chiffre --encrypt  
--recipient identificateur nom_de_fichier_clair
```

L'identificateur est l'identificateur d'une clef maître (un long numéro en hexadécimal ou l'adresse email). Pour déchiffrer un message, on utilise l'option **-decrypt** :

```
gpg --output nom_de_fichier_dechiffre --decrypt nom_de_fichier_chiffre
```

On peut aussi chiffrer des documents en utilisant une clef symétrique. Il faut pour cela donner un mot de passe à partir duquel gpg générera une clef symétrique. Toute personne connaissant ce mot de passe pourra déchiffrer le document :

```
gpg --output doc.gpg --symmetric doc
```

Rapport. Envoyez des fichiers aux personnes dont vous avez la clef publique de chiffrement, et déchiffrez les fichiers qu'on vous a envoyé. Illustrez ce que vous avez fait avec des captures d'écran.

Exercice 4 : Générer et vérifier des signatures

Une signature date et certifie l'authenticité d'un document. Si le document est modifié, la vérification de la signature va échouer.

L'option **-sign** est utilisée pour générer une signature numérique. Le document à signer est l'entrée et le document signé est la sortie. Le document est compressé avant d'être signé et la sortie est au format binaire.

```
gpg --output fichier_signe --sign fichier_a_signer
```

la vérification de la signature se fait avec l'option **verify**. L'option **decrypt** permet à la fois de vérifier et de récupérer le document.

```
gpg --output fichier_a_verifier --sign fichier_signe  
gpg --output fichier_a_verifier --decrypt fichier_signe
```

Un autre modèle de signature permet de signer un document sans le compresser en l'incluant dans un texte ASCII, ce qui est recommandé pour des mails, par exemple. L'option **-clearsign** permet de réaliser cette opération :

```
gpg --output fichier_signe --clearsign fichier_a_signer
```

Rapport. Envoyez des fichiers signés aux personnes qui ont votre clef publique de signature, et vérifiez les fichiers qu'on vous a envoyé. Illustrez ce que vous avez fait avec des captures d'écran.

Exercice 5 : Validation des clefs importées

On a déjà vu comment importer une clef, c'est-à-dire la stocker dans son trousseau de clefs. Une clef importée peut être utilisée suivant son type pour vérifier la signature de son propriétaire ou pour lui envoyer des messages chiffrés.

Mais on peut aussi valider une clef importée en créant un certificat indiquant que l'identité associée à la clef est correcte, et qu'on fait plus ou moins confiance à la personne propriétaire de cette clef pour valider d'autres clefs. Le but est de profiter aussi des identités qui auront été validées par ce propriétaire avec cette clef.

Procédure de validation d'une empreinte :

1. Soit avec la commande `fpr`, soit directement en lançant `gpg -fingerprint identifiant`, obtenir l'empreinte de la clef que vous avez importée ;
2. L'empreinte de la clef doit ensuite être vérifiée avec le propriétaire de la clé. Ce peut être fait en personne, au téléphone, ou par tout autre moyen, du moment que vous pouvez garantir que vous communiquez bien avec le vrai propriétaire de la clé. Si l'empreinte que vous obtenez est la même que celle que le propriétaire de la clé obtient, alors vous pouvez être sûr que vous avez une copie correcte de la clé.
3. Après l'étape 2, il faut éditer la clef (la signer avec `sign` et indiquer la confiance qu'on a en son propriétaire avec `trust`)

Signature d'une clef importée. Après avoir vérifié l'empreinte, vous pouvez signer la clé pour la valider. Cette procédure de validation est propagée aux autres utilisateurs en tenant compte de la confiance que vous avez en eux dans le *Web of Trust*. Il y a quatre niveaux de confiance en les autres autres propriétaires de clefs du *Web of Trust* :

unknown : On ne sait rien sur la façon dont le propriétaire signe les clés. Les clés de votre trousseau de clés publiques ont par défaut ce niveau de confiance ;

none : On sait que le propriétaire ne vérifie pas consciencieusement les clés avant de les signer ;

marginal : Le propriétaire comprend l'implication de la signature des clés et valide les clés avant de les signer

full : Le propriétaire comprend complètement les implications de la signature des clés, et fait très attention ;

ultimate : Vous considérez qu'une signature par ce propriétaire est équivalente à une signature par vous-même ;

Le niveau de confiance sur une clé est une chose personnelle que vous attribuez. Cette information est considérée comme privée. Ce n'est pas emballé avec la clé lorsque vous l'exportez ; elle est même enregistrée séparément de vos trousseaux, dans une base de données distincte. La commande `trust` est utilisée pour attribuer un niveau de confiance à une clé :

```
gpg --edit-key identificateur
...
trust
```

Si la clé est signée par vous-mêmes le niveau de validité est automatiquement *full*. La confiance est utilisée pour valider des clés qu'on ne peut valider personnellement.

Rapport. Signez vos clefs et mettez une relation de confiance sur les clefs que vous avez importées, et signez-les. Exportez une seconde fois les clefs connues. ¹

¹Dans la toile de confiance, le premier export correspond à dire votre état, et dans le second vous ajoutez ce que vous connaissez de vos voisins. Chaque fois qu'on apprend de nouvelles clefs, il faut exporter ses connaissances pour les propager

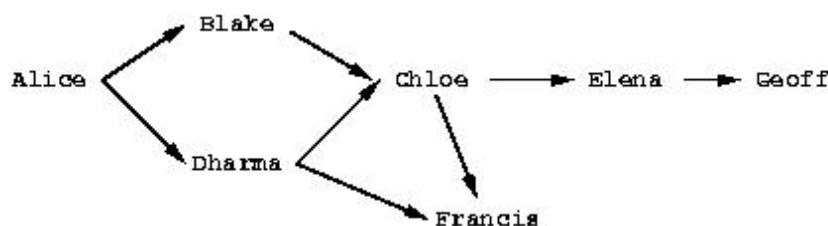


FIG. 2 : *Web of Trust*. Une flèche de *A* vers *B* indique que *A* a signé la clef publique de *B*.

Exercice 6 : *Web of Trust d'un sujet.*

La figure 2 montre un exemple de toile de confiance dont Alice est la racine. Le graphe illustre qui a signé les clés de qui. L'élaboration d'un réseau de confiance nécessite des algorithmes spécifiques pour valider les clés (la relation de confiance) et pour propager la relation de confiance des autres utilisateurs. Une clé est considérée comme valide si vous l'avez signée personnellement, ou si l'algorithme de propagation la calcule comme valide. L'algorithme utilisé dans le *Web of Trust* par défaut est le suivant (avec 3 clefs marginales au lieu de 2).

Clefs valides dans le Web of Trust : une clé *K* est considérée comme valide si elle remplit deux conditions :

1. Elle est signée par suffisamment de clés valides, c'est-à-dire si :
 - Vous l'avez signée personnellement ;
 - ou elle a été signée par une clé à laquelle vous accordez une confiance complète (*full trust*)
 - ou elle a été signée par 2 clés auxquelles vous accordez une confiance marginale (*marginal trust*)
2. Le chemin des clés signées conduisant de *K* jusqu'à votre propre clé mesure moins de cinq étapes.

Rapport. Reprenez le graphe de reconnaissance de la Fig. 2, puis étudiez la validité des clés, du point de vue d'Alice, avec les différents scénarios décrits dans le tableau ci-dessous :

Trust		Validity	
marginal	full	marginal	full
	Dharma		
Blake, Dharma			
Chloe, Dharma			
Blake, Chloe, Dharma			
	Blake, Chloe, Elena		

Rapport Exportez vos certificats pour les partager entre vous, et tentez de communiquer (en signant ou chiffrent des messages) avec des personnes dont vous n'avez pas directement signé la clef. Donnez aussi la liste des certificats (`-list-chain`) qui vous permettent d'utiliser les clefs que vous connaissez.